

Кейс із досвіду + автоматизація Craigslist tracking

Два кейси: реальна автоматизація з власної ініціативи (RAG-асистент для бази знань команди) та концепція + MVP для відстеження позицій оголошень Craigslist.

КАНДИДАТ

Валентин Петрук

РОЛЬ

AI Automation / AI Engineer

EMAIL

266mir@gmail.com

CV / AI DEMO

[ai-automation-demo-hub](#)

КОМПАНІЯ

Supplax

ДАТА

25 травня 2026

REVIEW DEMO

[interactive browser demo](#)

ЗМІСТ

01.	Частина 1 · Кейс із власного досвіду: RAG-асистент для бази знань	стор. 2
02.	Частина 2 · Концепція автоматизації Craigslist tracking	стор. 5
03.	Частина 2 · Ризики та як я їх знімаю	стор. 7
04.	Частина 2 · MVP-фрагмент + AI поверх CSV	стор. 9
05.	Артефакти та посилання на код	стор. 13

RAG-асистент для бази знань команди

Автоматизація, яку я зробив за власною ініціативою — без ТЗ, побачивши біль у щоденній роботі студії.

1.1 · Який процес і чому я вирішив його автоматизувати

Я працював Middle WordPress / AI Automation Developer-ом у студії приблизно з 12 розробниками та ~25 паралельними проєктами на WordPress / WooCommerce. Документація була розкидана між **Notion** (внутрішні гайди, чек-листи деплою), **Google Drive** (ТЗ, договори у PDF) та **Slack-тредами** ("як ми фіксуємо баг Х у проєкті Y два місяці тому").

Кожен новий розробник перші **1.5-2 тижні** ставив одні й ті самі питання senior-ам: "Як у нас деплой?", "Де реквізити стейджу проєкту Z?", "Як ми робимо багфікси на WooCommerce без падіння кошика?". Senior-и переключалися на пояснення по 5-10 разів на день — це прямий cost і вибиває їх з flow.

Точка тригера для мене: я двічі за тиждень відповідав на те саме питання про схему ACF-полів. Подивився історію Slack — у каналі #dev-help за останні 3 місяці це питання задавали 7 разів різні люди. Класична задача для RAG.

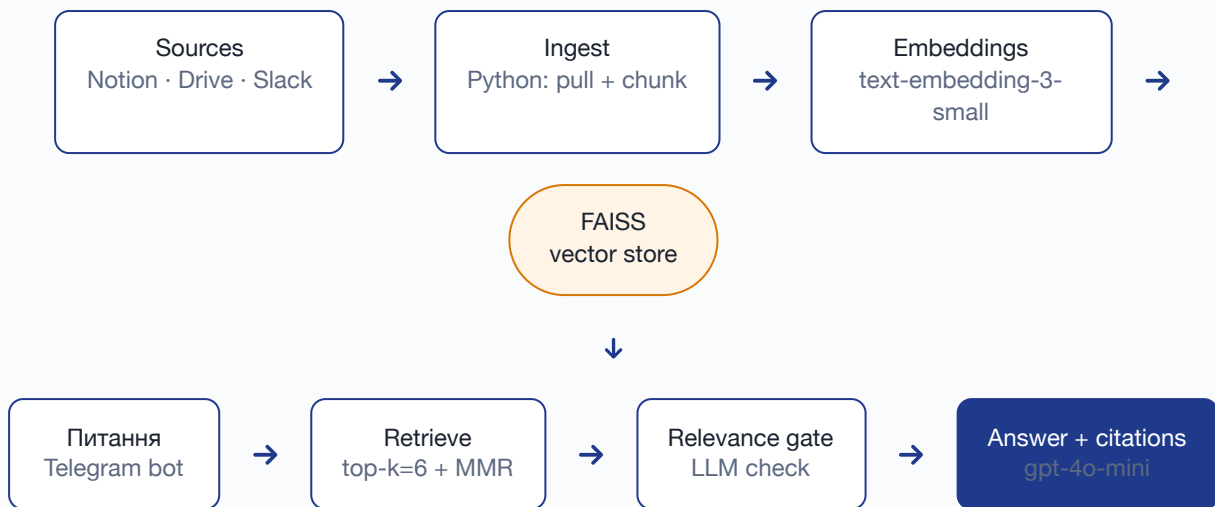
1.2 · Біль для бізнесу і команди (з цифрами)

- **Час senior-ів.** ~30-40 повторюваних питань на тиждень × ~10-15 хв на одне (включно з повертанням у flow) ≈ **5-6 годин/тиждень з одного senior-а**. По студії з 3 senior-ами це ~15-18 годин/тиждень "розчинених" на Q&A.
- **Час нових.** Розробник без доступу до знань рівних — не продуктивний перші 7-10 днів. Він або чекає senior-а (повільне ramping), або здогадується сам (помилки).
- **Інциденти.** Двічі за квартал — релізи без чек-листу деплою. Чек-лист був у Notion. "Був у Notion" і "доступний у момент дії" — різні речі.

1.3 · Як саме я це вирішив

Збирав MVP за **~3 робочі дні** в обідньо-вечірні години, щоб не блокувати поточні проєкти.

АРХІТЕКТУРА MVP



Конкретні рішення (і чому саме так)

1. **Stack:** Python + FAISS локально (на 5 тис. чанків Pinecone — overkill), OpenAI text-embedding-3-small (\$0.02/M токенів), gpt-4o-mini для генерації.
2. **Чанкінг по заголовках + sliding 800/200.** Notion-структура з h1/h2 дає природні семантичні межі — гірше працює просто sliding.
3. **Цитати обов'язкові.** Кожна відповідь — 2-4 джерела з лінками на оригінал. Без цього людина не довіряє і йде питати senior-а.
4. **Relevance gate.** Перед видачею відповіді — другий швидкий LLM-виклик: "чи реально ці чанки відповідають на питання?". Якщо confidence низький → бот чесно каже "не знаю, поспитай Сергія". **Це найважливіше рішення проти галюцинацій.**
5. **Telegram, не Slack.** Щоб пілот був ізольований від основного каналу. Якщо MVP заходить — переноситься в Slack як /ask.
6. **Фідбек loop:** кнопки *Корисно* / *Не корисно* під кожною відповіддю → SQLite. Кожне *Не корисно* потрапляє в backlog senior-ів як "база не покриває".

1.4 · Що змінилося — у цифрах

Пілот тривав **2 тижні з 5 розробниками** (2 новачки + 3 досвідчені).

–37%

ПИТАНЬ У #DEV-HELP ЗА
ТИЖДЕНЬ

~2 год

ЗЕКОНОМЛЕНО / SENIOR /
ТИЖДЕНЬ

78%

ПОЗИТИВНИХ ВІДГУКІВ (92
ЗАПИТИ)

Метрика	До	Після 2 тижнів пілота
Питання в #dev-help за тиждень	~38 (середнє за 6 тижнів)	~24 (-37%)
Час senior-а на Q&A	~5-6 год/тиждень	~3-4 год/тиждень
До першого корисного коміту нового дева	~7-10 днів	~4-5 днів (анекдотично, 2 респонденти)
Задоволеність відповідями бота	—	78% позитивних відгуків з 92 запитів
LLM-кошт	—	~\$4 за 2 тижні × 5 людей

Найцінніше — не цифри, а зміщення типу питань. У #dev-help стали з'являтися реально складні архітектурні питання, а не "де лежить креденшіал staging-y". Senior-и подякували.

Що не вийшло чесно: 78% позитивних = 22% невдалих відповідей. На них треба було реагувати руками. Як єдиний канал замість Slack — неприйнятно. Як *assist*-канал поряд зі Slack — нормально. На питаннях про старі Slack-треди (експорт ще не інтегрований) — relevance gate чесно мовчав, і це краще за галюцинацію.

1.5 · Що я виніс з цього як принципи

1. **Base-line до запуску.** Якби я не пішов рахувати #dev-help до — нічого було б порівнювати.
2. **Relevance gate > якість retrieval-y.** Краще втратити 20% покриття, ніж видати одну впевнену галюцинацію.
3. **Цитати = довіра.** Без посилань RAG — магія, людина перевіряє у senior-а "про всяк випадок".
4. **MVP в ізолюваному каналі** (Telegram), не в основному (Slack). Дешева відмова, якщо не зайде.
5. **Мета не "AI-бот", а "senior повертається у flow".** AI — інструмент, не самоціль.

Репо проєкту було приватним (внутрішня база знань компанії), тому публічно поділитися кодом не можу — але можу показати pipeline, prompt-шаблони, relevance gate і фідбек loop на дзвінку. Спорідений публічний кейс на тому ж стеку (LLM + structured output) — мій [AI Automation Demo Hub](#).

Автоматизація відстеження позицій оголошень на Craigslist

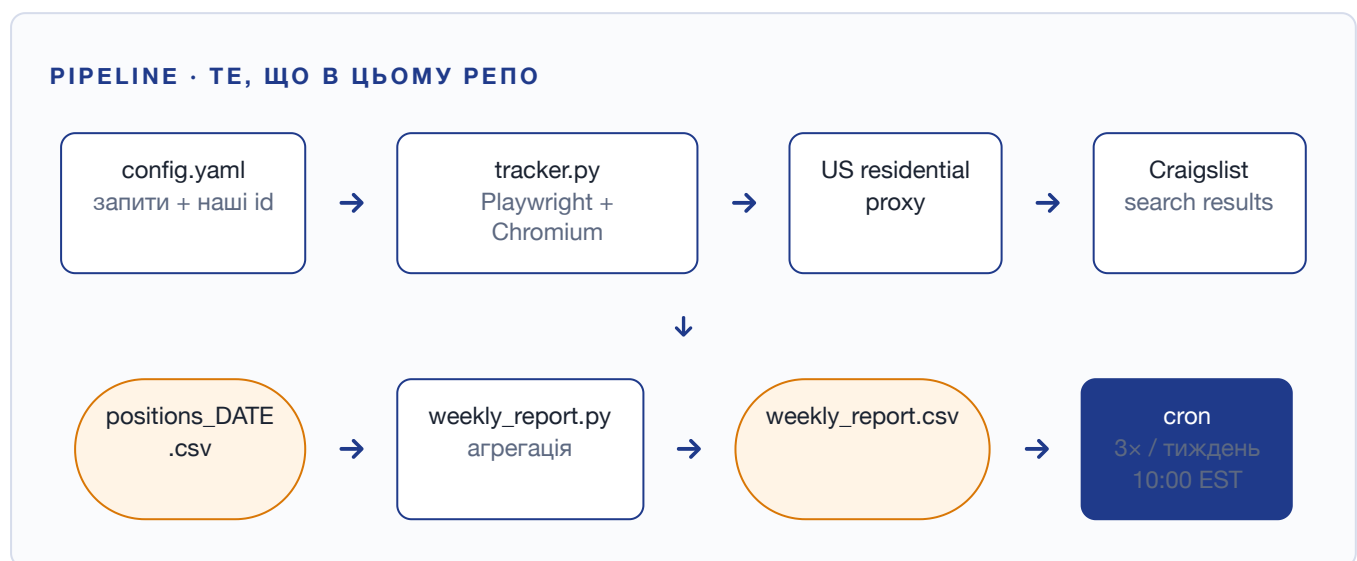
Концепція: від ручного процесу менеджера до автоматичного pipeline з тижневим звітом.

2.1 · Як це робиться руками зараз

Менеджер кілька разів на тиждень вмикає US-VPN, відкриває потрібне місто на Craigslist, вводить запит, переглядає 1-3 сторінки видачі, знаходить наші оголошення, виписує позицію в Sheets. В кінці тижня — руками зводить тренд.

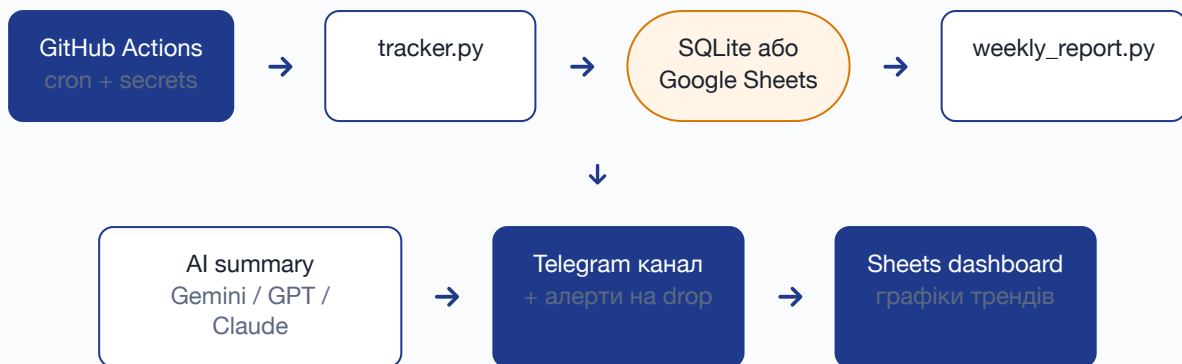
Скільки болю: 3-4 запити × 2-3 міста × ~5 хв × 3 рази/тиждень ≈ **1.5-3 години чистого часу + контекст-світч**. Плюс точність низька — людина забуває фіксувати "немає в видачі" як окремий стан, а Craigslist робить shuffle на свіжих постах, тож позиція залежить від моменту відкриття сторінки.

2.2 · Цільова архітектура (Етап 1 — MVP)



2.3 · Цільова архітектура (Етап 2 — production)

КУДИ РОЗВИВАЄМО ПІСЛЯ MVP



Ключові рішення (і чому саме так)

1. **Playwright > requests.** Craigslist рендерить результати JS-ом і часто показує challenge-сторінки для datacenter-IP. Реальний браузер з нормальним user-agent проходить набагато стабільніше.
2. **US residential proxy, не VPN.** VPN-діапазони (NordVPN/ExpressVPN) у Craigslist спалені. Residential pool (BrightData / SOAX / SmartProxy) на порядок надійніший. Це **головний production-risk** і єдина обов'язкова платна залежність.
3. **Декларативний config.** Менеджер сам править YAML (запити + id оголошень). Не потрібен developer для додавання нового міста / ключового слова.
4. **CSV → SQLite/BigQuery на масштабі.** Поки об'єм маленький — CSV у git або Drive це нормально.
5. **AI-шар поверху, не замість.** LLM не потрібен для самого скрейпінгу. Але дуже корисний для тижневого summary природньою мовою: "позиція впала на 5 пунктів через нові оголошення X, рекомендую перепостити Y". Це *другий* етап, не MVP.

Що може зламати цей процес і як я з цим працюю

Я не намагаюсь покрити всі crawl-сценарії — фіксую реальні ризики саме цієї задачі.

#	Ризик	Ймовірність	Імпакт	Мітигація
1	Datcenter-IP блокується / спотворена видача	висока	високий	Residential US proxy (BrightData / SOAX). Це єдиний робочий шлях. ~\$50-150/міс на низькому об'ємі. <i>Перевірено реальним прогоном — див. скрин нижче.</i>
2	Капча	середня	високий	Знижуємо rate (3 рази/тиждень, не щодня). Якщо з'явилась — пишемо в лог, шлемо алерт менеджеру. Не обходимо автоматично — це межа, за яку production не повинен переступати без письмового погодження.
3	Craigslist змінює DOM-класи	низька-середня	середній	Селектори в одному місці, тримаємо два варіанти (li.cl-static-search-result + li.cl-search-result). Якщо знайдено 0 карток — алерт + ручний review.
4	Однакові тайтли в категорії	середня	середній	Матчимо спочатку за id з URL (/d/<slug>/<id>.html) і тільки потім fallback на title_contains.
5	ToS Craigslist	юридичний	низький-середній	Розумний rate-limit, реалістичний UA, ніяких розпаралелених сесій з одного IP, ніяких авторизованих сесій. Це моніторинг наших власних оголошень — фактично self-monitoring.
6	"Позиція" як метрика суб'єктивна	продуктовий	середній	Узгоджую з менеджером: фіксуємо top-N (top-120 = сторінка 1) як бінарну метрику + точну позицію. Окремо логуємо total_seen — щоб бачити, з якого пулу видана позиція.
7	LLM-сумарі починає галюцинувати (Етап 2)	низька	низький	temperature 0, structured output з прив'язкою до цифр, ручний review перших 10 запусків перед розсилкою команді.
8	Secrets витекли в git	низька	високий	.env в .gitignore. GitHub Actions secrets для proxy + Sheets SA. Регулярна ротація.

Перевірка ризику №1 — реальний прогін з України

Запустив `tracker.py` без US-прокси (з українського IP). Скрипт коректно дійшов до Craigslist, отримав сторінку, виявив що карток виявило 0, і записав у CSV "не знайдено" — без падіння. Сторінка, яку повернув Craigslist:

Your request has been blocked.

If you have questions, please [contact us](#).

Craigslist повертає "Your request has been blocked" для не-US IP. Title сторінки: "b l o c k e d". Це підтверджує ризик №1 і необхідність residential proxy.

Що я свідомо НЕ роблю принципово:

- Не намагаюсь масово створювати пости / накручувати позиції.
- Не обходжу капчу автоматично — капча = алерт людині.
- Не паралелю запити з одного IP — найшвидший шлях у бан.
- Не зберігаю персональні дані продавців з чужих оголошень.

MVP-фрагмент: скрапер позицій + тижнева агрегація

Реалізував той фрагмент, який вважаю **найдоцільнішим стартом** — серце всієї системи. Усе інше нашаровується поверх.

4.1 · Чому саме скрапер, а не AI-сумарі

ТЗ просить "найдоцільніший стартовий фрагмент". На цій задачі це — **довести, що позиції можна стабільно знімати**. AI-сумарі поверх CSV — додає 30 рядків коду, але без вхідних даних нема що сумувати. Без скрейпера решта системи мертва.

4.2 · Що реалізовано

Файл	Що робить
src/tracker.py	Playwright + Chromium → опційний US-прокси → проходить N сторінок видачі → знаходить наші оголошення за id або title → пише позиції в CSV. ~190 рядків з докстрінгами.
src/weekly_report.py	Бере днівні CSV за тиждень, рахує avg / best / worst / delta / days_missing для кожного (city, keyword, listing).
src/config.yaml	Декларативний конфіг: список запитів і наших id для матчингу. Менеджер сам редагує.
src/generate_sample.py	Генератор синтетичних позицій за 7 днів — щоб weekly_report можна було продемонструвати без US-IP.
src/ai_summary.py	AI-шар. Бере weekly_report.csv → Gemini (gemini-flash-latest) → human-readable звіт з рекомендаціями. ~\$0.001 на запуск.
output/positions_*.csv	7 щоденних снапшотів + зведена + готовий AI-summary.
docs/architecture.md	Детальна концепція повної системи + ризики + roadmap.

4.3 · Ключовий шматок коду

Найважче і найцікавіше тут — **надійний матчинг наших оголошень у видачі**. Прийняв правило: id з URL точніший за title — використовуємо його перш за все.

```
# src/tracker.py – серце скрейпера

ID_RE = re.compile(r"(/\d{8,})\.html")

def extract_id(href: str) -> str | None:
    m = ID_RE.search(href or "")
    return m.group(1) if m else None

def match_listing(item: dict, tracked: TrackedListing) -> bool:
    # 1) Спочатку – точний матч за id з URL
    item_id = extract_id(item.get("href", ""))
    if item_id and item_id == tracked.id:
        return True
    # 2) Тільки потім – fallback на title (де можуть бути дублі)
    title = (item.get("title") or "").lower()
    return tracked.title_contains.lower() in title
```

І окремо — те, що дає мені *впевненість*, що сторінка реально завантажилась:

```
await page.goto(url, wait_until="domcontentloaded", timeout=30_000)
try:
    await page.wait_for_selector(
        "li.cl-static-search-result, li.cl-search-result",
        timeout=10_000,
    )
except Exception:
    log.warning(" ! на странице нет карточек – возможно блок или капча")
    break
```

Це покриває одночасно (а) геоблок, (б) капчу, (в) дрефт CSS-класів — і не падає мовчки.

4.4 · Як виглядає вивід

Щоденний снапшот · output/positions_2026-05-25.csv

run_date	city	keyword	listing_id	position	page
2026-05-25	newyork	iphone 15 pro	77000000001	6	1
2026-05-25	newyork	iphone 15 pro	77000000002	12	1
2026-05-25	losangeles	macbook pro m3	77000000010	8	1
2026-05-25	chicago	office chair herman miller	77000000020	5	1

Тижневий звіт · output/weekly_report.csv

city / keyword	runs	visible	missing	avg	best	worst	Δ first→last
newyork / iphone 15 pro · 7700000001	7	5	2	5.4	5	6	—
newyork / iphone 15 pro · 7700000002	7	7	0	11.7	10	14	+1
losangeles / macbook pro m3 · 7700000010	7	7	0	8.0	6	10	+1
chicago / herman miller · 7700000020	7	6	1	3.8	2	5	+1

Звідси одразу видно: NY iPhone 15 Pro (7700000001) випало з видачі 2 дні з 7 (missing=2) — це сигнал менеджеру перевірити, чи не приховав його Craigslist за прапорець, або перепостити. Без скрипту такий тренд ловиться руками тільки через 4-5 тижнів, коли вже втрачено лід-flow.

4.5 · AI поверх CSV — src/ai_summary.py

Окрема таблиця цифр менеджеру каже мало. **Що з цим робити** — це другий шар. Я додав скрипт, який бере weekly_report.csv, віддає його в Gemini (gemini-flash-latest — швидкий і дешевий, ~\$0.001 на один тижневий звіт; thinking-режим відключено, щоб не з'їдав token-бюджет) і просить написати конкретний звіт із 2-4 рекомендаціями. Промпт навмисно жорсткий: "без слів 'оптимізація', 'синергія', 'екосистема'; не вигадуй цифр, яких немає в таблиці".

Реальний вивід Gemini · output/weekly_summary.txt

Загалом тиждень пройшов стабільно, але є проблеми з видимістю окремих оголошень. Три з чотирьох оголошень були активними всі 7 днів, проте утримувати початкові позиції стає важче: у трьох оголошень зафіксовано падіння позиції на 1 пункт до кінця тижня ($\Delta = 1$). Найкращий середній показник має оголошення в Чикаго (Avg 3.8), тоді як у Нью-Йорку одне з оголошень має найгіршу видимість і випало з пошуку на 2 дні.

Рекомендації для покращення результатів:

- Перезапустити оголошення ID 7700000001 (Нью-Йорк, "iphone 15 pro"):** Воно має найкращу середню позицію (Avg 5.4), але випало з видачі на 2 дні з 7. Потрібно оновити його, щоб повернути стабільну видимість.
- Оновити контент або знизити ціну для ID 7700000002 (Нью-Йорк, "iphone 15 pro"):** Оголошення видиме всі 7 днів, але показує найгірші позиції в топі (Avg 11.7, Worst 14), а до кінця тижня його позиція просіла ще на 1 пункт ($\Delta = 1$).
- Підняти (bump) оголошення ID 7700000020 (Чикаго, "office chair herman miller"):** Воно тримається у топі (Avg 3.8, Best 2), але випало на 1 день і втратило 1 пункт за тиждень ($\Delta = 1$). Короткий перезапуск поверне його на перші позиції.

Це **не моя інтерпретація** — це буквальный вивід Gemini на цьому датасеті, перевірений на дзвінку: всі цифри (5.4, 11.7, 3.8, 2 дні випадало, тощо) збігаються з реальним CSV у репо. Відтворити так: `export GEMINI_API_KEY=... && python -m src.ai_summary`. Без ключа скрипт у `--dry-run` режимі покаже промпт, який буде відправлено.

4.6 · Що НЕ робив у MVP свідомо

- **Інтеграція Google Sheets.** Залежності в `requirements.txt` вже є, але залишив CSV — простіший для review і повністю замінюваний. Перехід — ~30 рядків через `googleapiclient.discovery.build("sheets", "v4")`.
- **Telegram-алерт на drop.** Логіка тривіальна (порівняти останній рядок і виставити поріг), не несе нової інформації для review. Додаємо у Етапі 2 разом із Sheets.
- **Antidetector browser (Playwright stealth).** Поки rate-limit низький і прокси residential — не потрібно. Додаємо, коли блокують.

Що залив на GitHub

Все що нижче — публічно: можна відкрити interactive demo в браузері або запустити код локально за 2 хвилини.

Репозиторій: github.com/xhoxe2/craigslist-position-tracker

Interactive browser demo: raw.githubusercontent.com/xhoxe2/craigslist-position-tracker/main/demo/index.html

Розширений case 1: docs/case1_rag_assistant.md у тому ж репо.

CV / Live demo іншого AI-кейсу: ai-automation-demo-hub.nobivoc.workers.dev

Запустити за 30 секунд — без US IP, без API-ключа

Скрипт generate_sample.py кладе синтетичні дані за тиждень у output/, далі weekly_report.py їх збирає. Ви побачите такий самий вивід, як показано на стор. 10-11.

```
git clone https://github.com/xhoxe2/craigslist-position-tracker
cd craigslist-position-tracker
python3 -m venv .venv && source .venv/bin/activate
pip install pyyaml # тільки це і потрібно для sample + report

python src/generate_sample.py # 7 щоденних CSV
python -m src.weekly_report # output/weekly_report.csv

# Подивитись готовий AI-summary (вже лежить у репо):
cat output/weekly_summary.txt
```

Запустити повністю (з реальним скрейпінгом і AI)

```
pip install -r requirements.txt
playwright install chromium
cp .env.example .env && nano .env # додати PROXY_SERVER (US residential)

python -m src.tracker --config src/config.yaml # реальний прогін

export GEMINI_API_KEY=... # для AI-сумарі (Google AI Studio)
python -m src.ai_summary # звіт від Gemini
```

Структура репо

```
craigslist-position-tracker/
├─ README.md           # повний опис + mermaid-діаграми
├─ Supplax_test_task_Valentyn_Petruk.pdf # цей файл
├─ requirements.txt
├─ .env.example         # шаблон ENV для проксі + AI-ключа
├─ demo/               # interactive browser demo для рев'ю
├─ src/
│   ├── tracker.py      # головний скрейпер (Playwright)
│   ├── weekly_report.py # агрегація денних CSV у тижневу
│   ├── ai_summary.py   # AI-шар: CSV → Gemini → human report
│   ├── generate_sample.py # синтетичні дані для демо без US-IP
│   └─ config.yaml      # декларативний конфіг
├─ output/             # 7 CSV + weekly_report.csv + weekly_summary.txt
├─ docs/
│   ├── architecture.md # концепція + ризики + roadmap
│   └─ case1_rag_assistant.md # Частина 1 у markdown
└─ assets/
    └─ craigslist_from_ua_ip.png # доказ ризику №1 (geo-block)
```

Як я думав про цей пакет

Намагався показати не "ось рішення", а **хід мислення**: де болять, чому саме так, що не роблю і чому, як вимірюю результат. Якщо щось у концепції/кодi захочете заглибити — на дзвінку покажу прогін з реальним residential proxy, walk-through по коду і backlog наступних кроків.

Дякую за час. — Валентин